notebook_apres_25_oct

November 3, 2025

[458]:
```
import pandas as pd
import numpy as np
from sklearn.model_selection import RandomizedSearchCV, train_test_split
from sklearn.ensemble import RandomForestClassifier,␣
 ↪GradientBoostingClassifier, VotingClassifier
from sklearn.linear_model import LogisticRegression
from sklearn.preprocessing import StandardScaler, OneHotEncoder
from sklearn.compose import ColumnTransformer
from sklearn.pipeline import Pipeline
from sklearn.metrics import (
    accuracy_score,
    recall_score,
    f1_score,
    classification_report,
    confusion_matrix,
    ConfusionMatrixDisplay
)
import matplotlib.pyplot as plt
from itertools import product
from typing import Dict, Any, Optional

print("Imports ok")
```

```
Imports ok
```

[459]:
```
train_df = pd.read_csv('data/train.csv', delimiter=';')

print(f"Forme des données d'entraînement : {train_df.shape}")
print(f"Nombre d'échantillons (n) : {train_df.shape[0]}")
print(f"Nombre de caractéristiques (d) : {train_df.shape[1] - 1}")  #␣
 ↪Soustraire 1 pour la colonne cible
print(f"Colonnes : {list(train_df.columns)}")
print("\nPremières lignes :")
print(train_df.head())

# Vérifier les types de données et informations de base
print("\nTypes de données et informations :")
print(train_df.info())
```

```
Forme des données d'entraînement : (4469, 15)
Nombre d'échantillons (n) : 4469
Nombre de caractéristiques (d) : 14
Colonnes : ['id', 'beer_style', 'bitterness_IBU', 'diacetyl_concentration',
'lactic_acid', 'final_gravity', 'sodium', 'free_CO2', 'dissolved_oxygen',
'original_gravity', 'pH', 'gypsum_level', 'alcohol_ABV',
'fermentation_strength', 'quality']

Premières lignes :
   id beer_style  bitterness_IBU  diacetyl_concentration  lactic_acid  \
0   0       Pale           7.997                   0.446        0.280
1   1       Pale           6.906                   0.222        0.303
2   2      Brown           7.183                   0.636        0.071
3   3       Pale           6.403                   0.242        0.260
4   4      Brown           7.917                   0.182        0.347

   final_gravity  sodium  free_CO2  dissolved_oxygen  original_gravity     pH  \
0         10.803   0.052    24.999           157.026             0.995  3.057
1          6.305   0.034    40.993           130.993             1.012  3.083
2          2.495   0.080    16.998            84.003             1.013  3.509
3          1.506   0.043    34.999           105.001             0.994  3.137
4          1.700   0.053     7.011            14.999             1.002  3.326

   gypsum_level  alcohol_ABV  fermentation_strength  quality
0         0.466        5.436                  9.545        7
1         0.476        5.137                  9.847        6
2         0.545        4.618                  8.771        5
3         0.308        5.909                 10.504        6
4         0.805        5.674                  9.748        7

Types de données et informations :
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 4469 entries, 0 to 4468
Data columns (total 15 columns):
 #   Column                  Non-Null Count  Dtype
---  ------                  --------------  -----
 0   id                      4469 non-null   int64
 1   beer_style              4469 non-null   object
 2   bitterness_IBU          4469 non-null   float64
 3   diacetyl_concentration  4469 non-null   float64
 4   lactic_acid             4469 non-null   float64
 5   final_gravity           4469 non-null   float64
 6   sodium                  4469 non-null   float64
 7   free_CO2                4469 non-null   float64
 8   dissolved_oxygen        4469 non-null   float64
 9   original_gravity        4469 non-null   float64
 10  pH                      4469 non-null   float64
 11  gypsum_level            4469 non-null   float64
```

```
 12  alcohol_ABV             4469 non-null   float64
 13  fermentation_strength   4469 non-null   float64
 14  quality                 4469 non-null   int64
dtypes: float64(12), int64(2), object(1)
memory usage: 523.8+ KB
None
```

[460]:
```
"""
Fonction de prétraitement pour créer de nouvelles caractéristiques basées sur␣
 ↪les caractéristiques existantes.
    Args:    processed_df (pd.DataFrame): Le DataFrame contenant les données␣
 ↪prétraitées.
    Returns:    pd.DataFrame: Le DataFrame avec les nouvelles caractéristiques␣
 ↪ajoutées.
"""
def preprocessing_caracterisques(processed_df: pd.DataFrame, is_train=True) ->␣
 ↪pd.DataFrame:
    df = processed_df.copy()

    df['free_CO2/dissolvedO2'] = df['free_CO2'] / (df['dissolved_oxygen'])
    df['gypmsum_level_x_sodium'] = df['gypsum_level'] * df['sodium']
    df['fermentation_strenght/alcohol_ABV'] = df['fermentation_strength'] /␣
 ↪(df['alcohol_ABV'])
    df['acid_x_gas'] = ((df['lactic_acid']) / (df['pH'])) * df['free_CO2/
 ↪dissolvedO2']
    df['log_gravity_drop'] = np.sqrt((df['original_gravity'] -␣
 ↪df['final_gravity']).clip(lower=0))
    df['bitterness_alcohol_ratio'] = df['bitterness_IBU'] / (df['alcohol_ABV'])

    return df
```

[461]:
```
"""
Fonction pour identifier les caractéristiques hautement corrélées.
    Args: df (pd.DataFrame): Le DataFrame contenant les données.
          t (float): Le seuil de corrélation au-dessus duquel les␣
 ↪caractéristiques sont considérées comme hautement corrélées.
    Returns: list: Une liste des noms des caractéristiques hautement corrélées.
"""
def high_corr(df: pd.DataFrame, t: float) -> list:
    # Calculer la matrice de corrélation absolue
    corr_matrix = df.corr(numeric_only=True).abs()
    upper = corr_matrix.where(np.triu(np.ones(corr_matrix.shape), 1).
 ↪astype(bool))
    # Trouver les caractéristiques avec une corrélation supérieure au seuil
    return [c for c in corr_matrix.columns if any(upper[c] > t)]
```

```
"""
Fonction pour identifier les caractéristiques faiblement corrélées avec la cible 'quality'.
    Args: df (pd.DataFrame): Le DataFrame contenant les données.
          t (float): Le seuil de corrélation en dessous duquel les caractéristiques
                             sont considérées comme faiblement corrélées.
    Returns:list: Une liste des noms des caractéristiques faiblement corrélées.
"""
def low_corr(df: pd.DataFrame, t: float) -> list:
    # Vérifier si la colonne 'quality' existe dans le DataFrame
    if 'quality' not in df:
        return []
    # Calculer la corrélation absolue avec la cible 'quality'
    c = df.corr(numeric_only=True)['quality'].abs()
    # Trouver les caractéristiques avec une corrélation inférieure au seuil
    return c[c < t].index.tolist()
```

[462]:
```
"""
Fonction pour supprimer les caractéristiques basées sur la corrélation.
    Args: df (pd.DataFrame): Le DataFrame contenant les données.
          high_corr_threshold (float)
          low_corr_threshold (float)
    Returns: pd.DataFrame: Le DataFrame avec les caractéristiques supprimées.
             list: Une liste des noms des caractéristiques supprimées.
"""
def drop_by_correlation(df: pd.DataFrame, high_threshold=0.9, low_threshold=0.05) -> tuple[pd.DataFrame, list]:
    # Identifier les caractéristiques à faible et haute corrélation
    low = low_corr(df, low_threshold)
    high = high_corr(df.drop(columns=['quality'], errors='ignore'), high_threshold)

    # Supprimer les caractéristiques identifiées
    features_to_drop = list(set(low + high))

    df = df.drop(columns=features_to_drop, errors='ignore')

    print("Features supprimées :", features_to_drop)

    return df, features_to_drop
```

[463]:
```
"""
Fonction pour imputer les valeurs manquantes dans les colonnes 'lactic_acid' et 'sodium'.
    Args: df (pd.DataFrame): Le DataFrame contenant les données.
```

```
        is_train (bool): Indique si le DataFrame est pour l'entraînement ou le test.
        stats (Optional[Dict[str, Any]]): Statistiques d'imputation pour le jeu de test.
    Returns: pd.DataFrame: Le DataFrame avec les valeurs imputées.
             Optional[Dict[str, Any]]: Statistiques d'imputation pour le jeu d'entraînement.
"""
def impute_values(df: pd.DataFrame,  is_train: True, stats: Optional[Dict[str, Any]] = None)  -> tuple[pd.DataFrame, Optional[Dict[str, Any]]]:

    dF = df.copy()

    cols = ['lactic_acid', 'sodium']

    for col in cols:
            dF.loc[dF[col] < 0, col] = np.nan

    if is_train:

        mean = {}

        global_means = dF[cols].mean().to_dict()
        mean['global_means'] = global_means

        dF = dF.fillna(dF.groupby('quality')[cols].transform('mean'))
        dF = dF.fillna(global_means)

        return dF, mean


    dF = dF.fillna(stats['global_means'])

    return dF, None
```

[464]:
```
def visualize_distributions(df: pd.DataFrame) -> None:
    corr = df.corr(numeric_only=True).abs()
    sns.heatmap(corr, cmap='coolwarm', center=0)
    sns.pairplot(df[['lactic_acid','pH','alcohol_ABV','final_gravity','quality']], hue='quality')
    plt.figure(figsize=(10,6))
    sns.heatmap(df[['gravity_drop','acid_balance','gas_balance','quality']].corr(),
                annot=True, cmap='coolwarm', center=0)
    plt.title("Corrélations des nouvelles features avec la qualité")
    plt.show()
```

```
    corr2 = df[['acid_x_gas','log_gravity_drop','quality']].corr()
    sns.heatmap(corr2, annot=True, cmap='coolwarm', center=0)
    plt.title("Corrélations des features non linéaires avec la qualité")
    plt.show()
```

[465]:
```
def gscore(y_val, y_pred_final, model):
    print(f"Accuracy: {accuracy_score(y_val, y_pred_final):.4f}")
    print(f"Recall Macro: {recall_score(y_val, y_pred_final, average='macro'):.
 ↪4f}")
    print(f"F1 Macro: {f1_score(y_val, y_pred_final, average='macro'):.4f}")

    print("\n" + classification_report(y_val, y_pred_final))

    cm = confusion_matrix(y_val, y_pred_final)
    disp = ConfusionMatrixDisplay(confusion_matrix=cm, display_labels=model.
 ↪classes_)
    disp.plot(cmap='Blues', values_format='d')
    plt.tight_layout()
    plt.show()

def training_score(model, y_val, y_pred):
    print(f"Meilleurs hyperparam {model.__class__.__name__} :{model.
 ↪best_params_}")
    print(f"Meilleur score en CV: {model.best_score_}")

    print(f"Accuracy sur validation: {accuracy_score(y_val, y_pred)}")
    print(f"Recall macro sur validation: {recall_score(y_val, y_pred,␣
 ↪average="macro")}")
```

[466]:
```
"""
1. On conserve seulement les classes 4 à 8, compte tenu les autres classes sont␣
 ↪très minoritaires ou absentes.
2. On crée des nouvelles features en combinant les features existants
2. On empute les valeurs absentes et incohérentes
3. On drop les features peu pertinents et redondants
"""
def preprossesing_dataset(train_df: pd.DataFrame) -> tuple[pd.DataFrame, list,␣
 ↪list]:

    df = train_df[train_df['quality'].between(4,8)]

    ## Decommenter pour voir les corrélations entre les features et la qualité
    #visualize_distributions(df)

    df = preprocessing_caracterisques(df, is_train=True)
    df, stats = impute_values(df, is_train=True)
```

```
    df, dropped_features = drop_by_correlation(
        df,
        high_threshold=0.95,
        low_threshold=0.02
    )


    return df, stats, dropped_features
```

[467]:
```
rl_df, stats, dropped_features = preprossesing_dataset(train_df)

if 'id' in rl_df.columns:
    rl_df = rl_df.drop(columns=['id'])

X = rl_df.drop(columns=['quality'])
y = rl_df['quality']

X_train, X_val, y_train, y_val = train_test_split(
    X, y, test_size=0.2, stratify=y, random_state=42
)

categorical_features = ['beer_style']
numeric_features = [col for col in rl_df.columns if col not in␣
 ↪categorical_features + ['quality']]

preprocessor = ColumnTransformer(transformers=[
    ('cat', OneHotEncoder(handle_unknown='ignore'), categorical_features),
    ('num', StandardScaler(), numeric_features)
])
```

```
Features supprimées : ['pH']
```

[468]:
```
"""
Optimisation de Random Forest
1. RandomizedSearchCV -> Effectue une recherche aléatoire des hyperparamètres,␣
 ↪on évalue avec 'recall_macro' sur X_train et y_train en validation croisée
2. On évalue les performances sur X_val
"""


rf_pipeline = Pipeline([
    ('preprocessor', preprocessor),
    ('classifier', RandomForestClassifier(random_state=42,␣
 ↪class_weight='balanced'))
])

rf_hyperparam = {
```

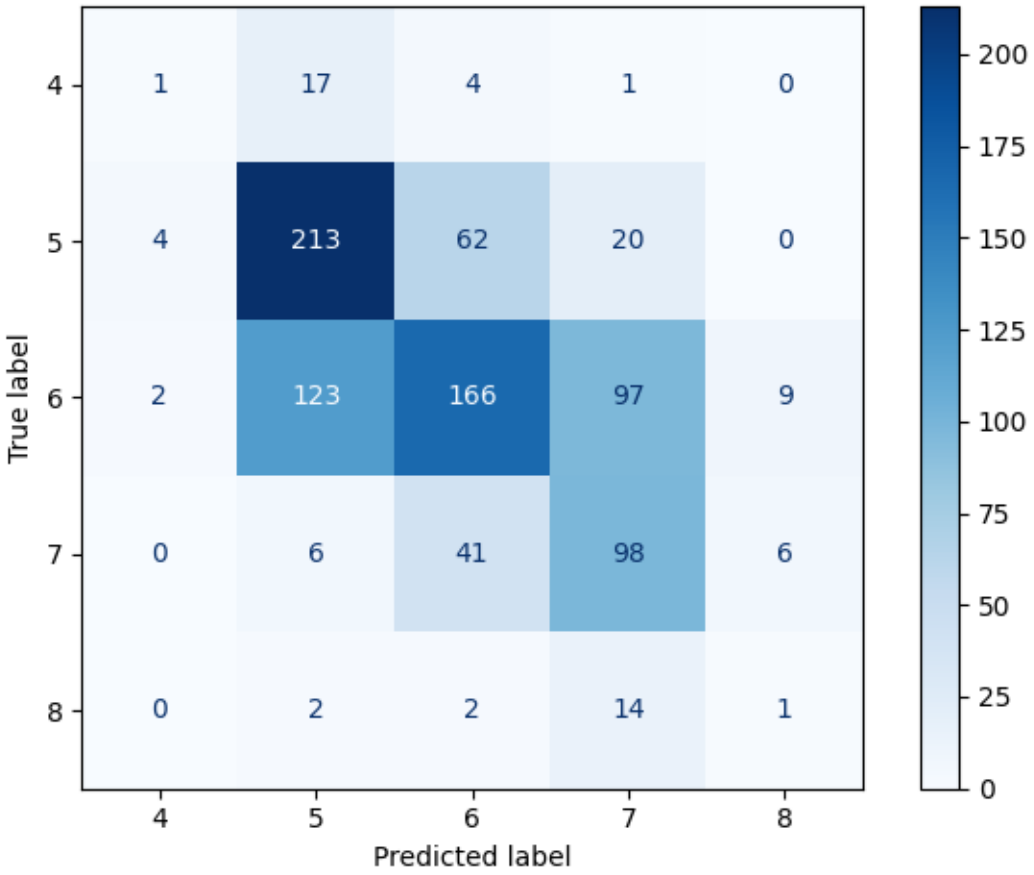
```
    'classifier__n_estimators': [400, 500, 600],
    'classifier__max_depth': [10, 15, 20, 25, None],
    'classifier__min_samples_split': [5, 10, 15],
    'classifier__min_samples_leaf': [1, 2, 3],
    'classifier__max_features': ['sqrt', None],
    'classifier__bootstrap': [True, False]
}

rf = RandomizedSearchCV (
    rf_pipeline,
    rf_hyperparam,
    n_iter=30,
    cv=5,
    scoring = 'recall_macro',
    n_jobs=-1,
    random_state=42
)

rf.fit(X_train, y_train)
y_pred_rf = rf.predict(X_val)
gscore(y_val, y_pred_rf, rf)
training_score(rf, y_val, y_pred_rf)
```

```
Accuracy: 0.5388
Recall Macro: 0.3751
F1 Macro: 0.3555
```

	precision	recall	f1-score	support
4	0.14	0.04	0.07	23
5	0.59	0.71	0.65	299
6	0.60	0.42	0.49	397
7	0.43	0.65	0.51	151
8	0.06	0.05	0.06	19
accuracy			0.54	889
macro avg	0.37	0.38	0.36	889
weighted avg	0.55	0.54	0.53	889




```
Meilleurs hyperparam RandomizedSearchCV :{'classifier__n_estimators': 600,
'classifier__min_samples_split': 5, 'classifier__min_samples_leaf': 1,
'classifier__max_features': 'sqrt', 'classifier__max_depth': 10,
'classifier__bootstrap': False}
Meilleur score en CV: 0.43649409525319466
Accuracy sur validation: 0.5388076490438695
Recall macro sur validation: 0.37512541288488455
```

[469]:
```
print("Optimisation gradient boosting...")

gb_pipeline = Pipeline([
    ('preprocessor', preprocessor),
    ('classifier', GradientBoostingClassifier(random_state=42))
])

gb_hyperparam = {
    'classifier__n_estimators': [100, 200, 300, 500],
    'classifier__learning_rate': [0.01, 0.03, 0.05, 0.1],
    'classifier__max_depth': [3, 5, 7, 9],
```

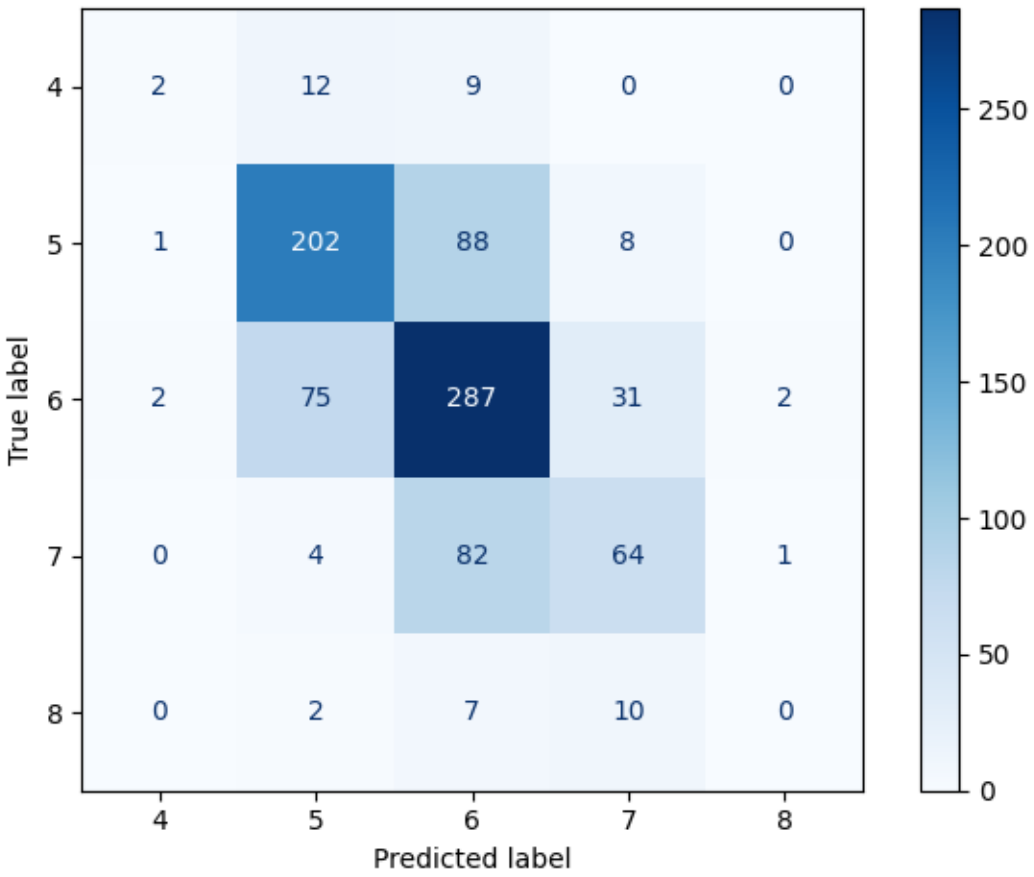
```
    'classifier__min_samples_split': [5, 10, 15, 20],
    'classifier__min_samples_leaf': [1, 2, 4],
    'classifier__subsample': [0.7, 0.8, 0.9, 1.0],
    'classifier__max_features': ['sqrt', 'log2', None]
}

gb = RandomizedSearchCV(
    gb_pipeline,
    gb_hyperparam,
    n_iter=30,
    cv=5,
    scoring='recall_macro',
    n_jobs=-1,
    random_state=42
)

gb.fit(X_train, y_train)
y_pred_gb = gb.predict(X_val)
gscore(y_val, y_pred_gb, gb)
training_score(gb, y_val, y_pred_gb)
```

```
Optimisation gradient boosting…
Accuracy: 0.6243
Recall Macro: 0.3819
F1 Macro: 0.3935
```

	precision	recall	f1-score	support
4	0.40	0.09	0.14	23
5	0.68	0.68	0.68	299
6	0.61	0.72	0.66	397
7	0.57	0.42	0.48	151
8	0.00	0.00	0.00	19
accuracy			0.62	889
macro avg	0.45	0.38	0.39	889
weighted avg	0.61	0.62	0.61	889




```
Meilleurs hyperparam RandomizedSearchCV :{'classifier__subsample': 1.0,
'classifier__n_estimators': 500, 'classifier__min_samples_split': 5,
'classifier__min_samples_leaf': 4, 'classifier__max_features': None,
'classifier__max_depth': 9, 'classifier__learning_rate': 0.1}
Meilleur score en CV: 0.38813696385778806
Accuracy sur validation: 0.6242969628796401
Recall macro sur validation: 0.38186095599607967
```

[470]:
```
print("Optimisation regression logistique...")

lr_pipeline = Pipeline([
    ('preprocessor', preprocessor),
    ('scaler', StandardScaler(with_mean=False)),
    ('classifier', LogisticRegression(
        solver='saga',
        max_iter=3000,
        random_state=42,
        class_weight='balanced',
        penalty='l2'
```

```
    ))
])

lr_hyperparam = {
    'classifier__C': [0.001, 0.01, 0.05, 0.1, 0.5, 1, 5, 10],
    'classifier__penalty': ['l1', 'l2', 'elasticnet'],
    'classifier__l1_ratio': [0, 0.25, 0.5, 0.75, 1.0]
}

lr = RandomizedSearchCV(
    lr_pipeline,
    lr_hyperparam,
    n_iter=20,
    cv=5,
    scoring='recall_macro',
    n_jobs=-1,
    random_state=42
)


lr.fit(X_train, y_train)
y_pred_lr = lr.predict(X_val)

gscore(y_val, y_pred_lr, lr)
training_score(lr, y_val, y_pred_lr)
```

```
Optimisation regression logistique…
```

```
/Users/yamira.poldosilva/Documents/UDEM/A25/IFT3395/kaggle_1_25-
10/.venv/lib/python3.13/site-packages/sklearn/linear_model/_logistic.py:1221:
UserWarning: l1_ratio parameter is only used when penalty is 'elasticnet'. Got
(penalty=l2)
  warnings.warn(
/Users/yamira.poldosilva/Documents/UDEM/A25/IFT3395/kaggle_1_25-
10/.venv/lib/python3.13/site-packages/sklearn/linear_model/_logistic.py:1221:
UserWarning: l1_ratio parameter is only used when penalty is 'elasticnet'. Got
(penalty=l2)
  warnings.warn(
/Users/yamira.poldosilva/Documents/UDEM/A25/IFT3395/kaggle_1_25-
10/.venv/lib/python3.13/site-packages/sklearn/linear_model/_logistic.py:1221:
UserWarning: l1_ratio parameter is only used when penalty is 'elasticnet'. Got
(penalty=l2)
  warnings.warn(
/Users/yamira.poldosilva/Documents/UDEM/A25/IFT3395/kaggle_1_25-
10/.venv/lib/python3.13/site-packages/sklearn/linear_model/_logistic.py:1221:
UserWarning: l1_ratio parameter is only used when penalty is 'elasticnet'. Got
(penalty=l2)
  warnings.warn(
```

```
/Users/yamira.poldosilva/Documents/UDEM/A25/IFT3395/kaggle_1_25-
10/.venv/lib/python3.13/site-packages/sklearn/linear_model/_logistic.py:1221:
UserWarning: l1_ratio parameter is only used when penalty is 'elasticnet'. Got
(penalty=l2)
  warnings.warn(
/Users/yamira.poldosilva/Documents/UDEM/A25/IFT3395/kaggle_1_25-
10/.venv/lib/python3.13/site-packages/sklearn/linear_model/_logistic.py:1221:
UserWarning: l1_ratio parameter is only used when penalty is 'elasticnet'. Got
(penalty=l2)
  warnings.warn(
/Users/yamira.poldosilva/Documents/UDEM/A25/IFT3395/kaggle_1_25-
10/.venv/lib/python3.13/site-packages/sklearn/linear_model/_logistic.py:1221:
UserWarning: l1_ratio parameter is only used when penalty is 'elasticnet'. Got
(penalty=l2)
  warnings.warn(
/Users/yamira.poldosilva/Documents/UDEM/A25/IFT3395/kaggle_1_25-
10/.venv/lib/python3.13/site-packages/sklearn/linear_model/_logistic.py:1221:
UserWarning: l1_ratio parameter is only used when penalty is 'elasticnet'. Got
(penalty=l2)
  warnings.warn(
/Users/yamira.poldosilva/Documents/UDEM/A25/IFT3395/kaggle_1_25-
10/.venv/lib/python3.13/site-packages/sklearn/linear_model/_logistic.py:1221:
UserWarning: l1_ratio parameter is only used when penalty is 'elasticnet'. Got
(penalty=l2)
  warnings.warn(
/Users/yamira.poldosilva/Documents/UDEM/A25/IFT3395/kaggle_1_25-
10/.venv/lib/python3.13/site-packages/sklearn/linear_model/_logistic.py:1221:
UserWarning: l1_ratio parameter is only used when penalty is 'elasticnet'. Got
(penalty=l2)
  warnings.warn(
/Users/yamira.poldosilva/Documents/UDEM/A25/IFT3395/kaggle_1_25-
10/.venv/lib/python3.13/site-packages/sklearn/linear_model/_logistic.py:1221:
UserWarning: l1_ratio parameter is only used when penalty is 'elasticnet'. Got
(penalty=l2)
  warnings.warn(
/Users/yamira.poldosilva/Documents/UDEM/A25/IFT3395/kaggle_1_25-
10/.venv/lib/python3.13/site-packages/sklearn/linear_model/_logistic.py:1221:
UserWarning: l1_ratio parameter is only used when penalty is 'elasticnet'. Got
(penalty=l2)
  warnings.warn(
/Users/yamira.poldosilva/Documents/UDEM/A25/IFT3395/kaggle_1_25-
10/.venv/lib/python3.13/site-packages/sklearn/linear_model/_logistic.py:1221:
UserWarning: l1_ratio parameter is only used when penalty is 'elasticnet'. Got
(penalty=l2)
  warnings.warn(
/Users/yamira.poldosilva/Documents/UDEM/A25/IFT3395/kaggle_1_25-
10/.venv/lib/python3.13/site-packages/sklearn/linear_model/_logistic.py:1221:
UserWarning: l1_ratio parameter is only used when penalty is 'elasticnet'. Got
```

```
(penalty=l2)
  warnings.warn(
/Users/yamira.poldosilva/Documents/UDEM/A25/IFT3395/kaggle_1_25-
10/.venv/lib/python3.13/site-packages/sklearn/linear_model/_logistic.py:1221:
UserWarning: l1_ratio parameter is only used when penalty is 'elasticnet'. Got
(penalty=l2)
  warnings.warn(
/Users/yamira.poldosilva/Documents/UDEM/A25/IFT3395/kaggle_1_25-
10/.venv/lib/python3.13/site-packages/sklearn/linear_model/_logistic.py:1221:
UserWarning: l1_ratio parameter is only used when penalty is 'elasticnet'. Got
(penalty=l2)
  warnings.warn(
/Users/yamira.poldosilva/Documents/UDEM/A25/IFT3395/kaggle_1_25-
10/.venv/lib/python3.13/site-packages/sklearn/linear_model/_logistic.py:1221:
UserWarning: l1_ratio parameter is only used when penalty is 'elasticnet'. Got
(penalty=l2)
  warnings.warn(
/Users/yamira.poldosilva/Documents/UDEM/A25/IFT3395/kaggle_1_25-
10/.venv/lib/python3.13/site-packages/sklearn/linear_model/_logistic.py:1221:
UserWarning: l1_ratio parameter is only used when penalty is 'elasticnet'. Got
(penalty=l2)
  warnings.warn(
/Users/yamira.poldosilva/Documents/UDEM/A25/IFT3395/kaggle_1_25-
10/.venv/lib/python3.13/site-packages/sklearn/linear_model/_logistic.py:1221:
UserWarning: l1_ratio parameter is only used when penalty is 'elasticnet'. Got
(penalty=l2)
  warnings.warn(
/Users/yamira.poldosilva/Documents/UDEM/A25/IFT3395/kaggle_1_25-
10/.venv/lib/python3.13/site-packages/sklearn/linear_model/_logistic.py:1221:
UserWarning: l1_ratio parameter is only used when penalty is 'elasticnet'. Got
(penalty=l2)
  warnings.warn(
/Users/yamira.poldosilva/Documents/UDEM/A25/IFT3395/kaggle_1_25-
10/.venv/lib/python3.13/site-packages/sklearn/linear_model/_logistic.py:1221:
UserWarning: l1_ratio parameter is only used when penalty is 'elasticnet'. Got
(penalty=l2)
  warnings.warn(
/Users/yamira.poldosilva/Documents/UDEM/A25/IFT3395/kaggle_1_25-
10/.venv/lib/python3.13/site-packages/sklearn/linear_model/_logistic.py:1221:
UserWarning: l1_ratio parameter is only used when penalty is 'elasticnet'. Got
(penalty=l2)
  warnings.warn(
/Users/yamira.poldosilva/Documents/UDEM/A25/IFT3395/kaggle_1_25-
10/.venv/lib/python3.13/site-packages/sklearn/linear_model/_logistic.py:1221:
UserWarning: l1_ratio parameter is only used when penalty is 'elasticnet'. Got
(penalty=l2)
  warnings.warn(
/Users/yamira.poldosilva/Documents/UDEM/A25/IFT3395/kaggle_1_25-
```

```
10/.venv/lib/python3.13/site-packages/sklearn/linear_model/_logistic.py:1221:
UserWarning: l1_ratio parameter is only used when penalty is 'elasticnet'. Got
(penalty=l2)
  warnings.warn(
/Users/yamira.poldosilva/Documents/UDEM/A25/IFT3395/kaggle_1_25-
10/.venv/lib/python3.13/site-packages/sklearn/linear_model/_logistic.py:1221:
UserWarning: l1_ratio parameter is only used when penalty is 'elasticnet'. Got
(penalty=l2)
  warnings.warn(
/Users/yamira.poldosilva/Documents/UDEM/A25/IFT3395/kaggle_1_25-
10/.venv/lib/python3.13/site-packages/sklearn/linear_model/_logistic.py:1221:
UserWarning: l1_ratio parameter is only used when penalty is 'elasticnet'. Got
(penalty=l2)
  warnings.warn(
/Users/yamira.poldosilva/Documents/UDEM/A25/IFT3395/kaggle_1_25-
10/.venv/lib/python3.13/site-packages/sklearn/linear_model/_logistic.py:1221:
UserWarning: l1_ratio parameter is only used when penalty is 'elasticnet'. Got
(penalty=l1)
  warnings.warn(
/Users/yamira.poldosilva/Documents/UDEM/A25/IFT3395/kaggle_1_25-
10/.venv/lib/python3.13/site-packages/sklearn/linear_model/_logistic.py:1221:
UserWarning: l1_ratio parameter is only used when penalty is 'elasticnet'. Got
(penalty=l2)
  warnings.warn(
/Users/yamira.poldosilva/Documents/UDEM/A25/IFT3395/kaggle_1_25-
10/.venv/lib/python3.13/site-packages/sklearn/linear_model/_logistic.py:1221:
UserWarning: l1_ratio parameter is only used when penalty is 'elasticnet'. Got
(penalty=l2)
  warnings.warn(
/Users/yamira.poldosilva/Documents/UDEM/A25/IFT3395/kaggle_1_25-
10/.venv/lib/python3.13/site-packages/sklearn/linear_model/_logistic.py:1221:
UserWarning: l1_ratio parameter is only used when penalty is 'elasticnet'. Got
(penalty=l2)
  warnings.warn(
/Users/yamira.poldosilva/Documents/UDEM/A25/IFT3395/kaggle_1_25-
10/.venv/lib/python3.13/site-packages/sklearn/linear_model/_logistic.py:1221:
UserWarning: l1_ratio parameter is only used when penalty is 'elasticnet'. Got
(penalty=l1)
  warnings.warn(
/Users/yamira.poldosilva/Documents/UDEM/A25/IFT3395/kaggle_1_25-
10/.venv/lib/python3.13/site-packages/sklearn/linear_model/_logistic.py:1221:
UserWarning: l1_ratio parameter is only used when penalty is 'elasticnet'. Got
(penalty=l1)
  warnings.warn(
/Users/yamira.poldosilva/Documents/UDEM/A25/IFT3395/kaggle_1_25-
10/.venv/lib/python3.13/site-packages/sklearn/linear_model/_logistic.py:1221:
UserWarning: l1_ratio parameter is only used when penalty is 'elasticnet'. Got
(penalty=l2)
  warnings.warn(
/Users/yamira.poldosilva/Documents/UDEM/A25/IFT3395/kaggle_1_25-
10/.venv/lib/python3.13/site-packages/sklearn/linear_model/_logistic.py:1221:
UserWarning: l1_ratio parameter is only used when penalty is 'elasticnet'. Got
(penalty=l2)
```

```
  warnings.warn(
/Users/yamira.poldosilva/Documents/UDEM/A25/IFT3395/kaggle_1_25-
10/.venv/lib/python3.13/site-packages/sklearn/linear_model/_logistic.py:1221:
UserWarning: l1_ratio parameter is only used when penalty is 'elasticnet'. Got
(penalty=l1)
  warnings.warn(
/Users/yamira.poldosilva/Documents/UDEM/A25/IFT3395/kaggle_1_25-
10/.venv/lib/python3.13/site-packages/sklearn/linear_model/_logistic.py:1221:
UserWarning: l1_ratio parameter is only used when penalty is 'elasticnet'. Got
(penalty=l1)
  warnings.warn(
/Users/yamira.poldosilva/Documents/UDEM/A25/IFT3395/kaggle_1_25-
10/.venv/lib/python3.13/site-packages/sklearn/linear_model/_logistic.py:1221:
UserWarning: l1_ratio parameter is only used when penalty is 'elasticnet'. Got
(penalty=l1)
  warnings.warn(
/Users/yamira.poldosilva/Documents/UDEM/A25/IFT3395/kaggle_1_25-
10/.venv/lib/python3.13/site-packages/sklearn/linear_model/_logistic.py:1221:
UserWarning: l1_ratio parameter is only used when penalty is 'elasticnet'. Got
(penalty=l1)
  warnings.warn(
/Users/yamira.poldosilva/Documents/UDEM/A25/IFT3395/kaggle_1_25-
10/.venv/lib/python3.13/site-packages/sklearn/linear_model/_logistic.py:1221:
UserWarning: l1_ratio parameter is only used when penalty is 'elasticnet'. Got
(penalty=l1)
  warnings.warn(
/Users/yamira.poldosilva/Documents/UDEM/A25/IFT3395/kaggle_1_25-
10/.venv/lib/python3.13/site-packages/sklearn/linear_model/_logistic.py:1221:
UserWarning: l1_ratio parameter is only used when penalty is 'elasticnet'. Got
(penalty=l1)
  warnings.warn(
/Users/yamira.poldosilva/Documents/UDEM/A25/IFT3395/kaggle_1_25-
10/.venv/lib/python3.13/site-packages/sklearn/linear_model/_logistic.py:1221:
UserWarning: l1_ratio parameter is only used when penalty is 'elasticnet'. Got
(penalty=l1)
  warnings.warn(
/Users/yamira.poldosilva/Documents/UDEM/A25/IFT3395/kaggle_1_25-
10/.venv/lib/python3.13/site-packages/sklearn/linear_model/_logistic.py:1221:
UserWarning: l1_ratio parameter is only used when penalty is 'elasticnet'. Got
(penalty=l1)
  warnings.warn(
/Users/yamira.poldosilva/Documents/UDEM/A25/IFT3395/kaggle_1_25-
10/.venv/lib/python3.13/site-packages/sklearn/linear_model/_logistic.py:1221:
UserWarning: l1_ratio parameter is only used when penalty is 'elasticnet'. Got
(penalty=l2)
  warnings.warn(
/Users/yamira.poldosilva/Documents/UDEM/A25/IFT3395/kaggle_1_25-
10/.venv/lib/python3.13/site-packages/sklearn/linear_model/_logistic.py:1221:
UserWarning: l1_ratio parameter is only used when penalty is 'elasticnet'. Got
(penalty=l2)
  warnings.warn(
/Users/yamira.poldosilva/Documents/UDEM/A25/IFT3395/kaggle_1_25-
10/.venv/lib/python3.13/site-packages/sklearn/linear_model/_logistic.py:1221:
```

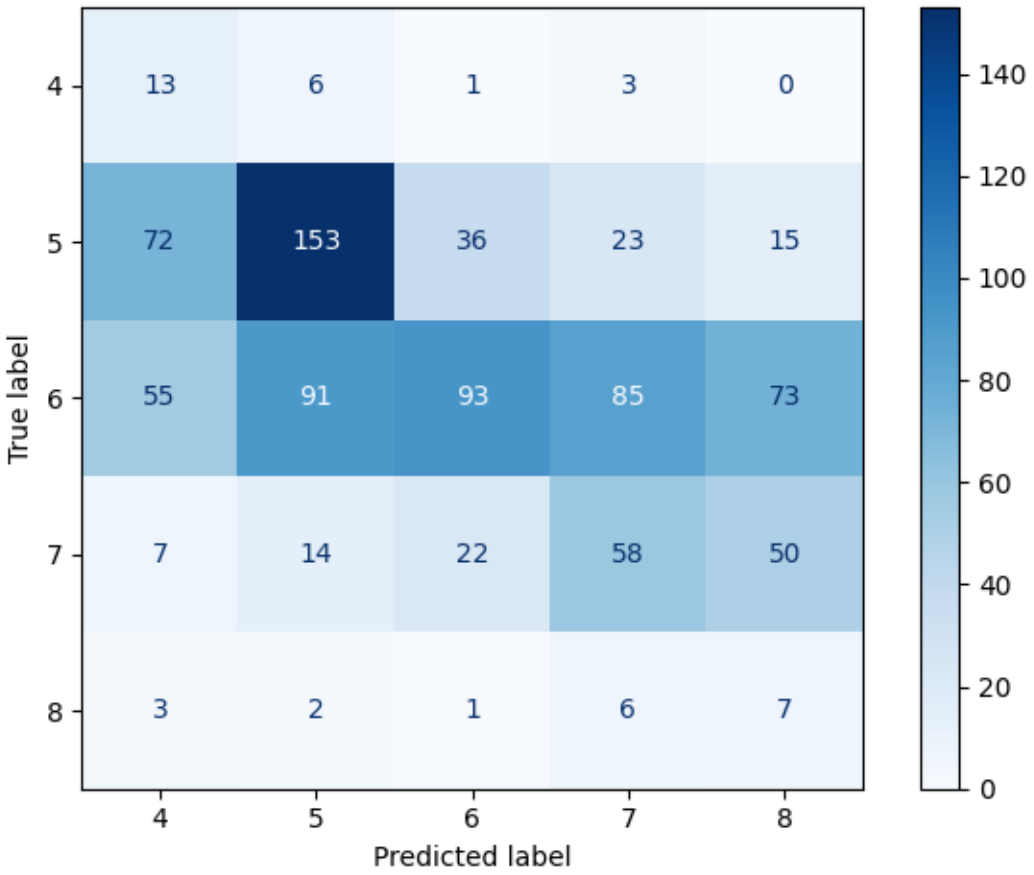
```
UserWarning: l1_ratio parameter is only used when penalty is 'elasticnet'. Got
(penalty=l2)
  warnings.warn(
/Users/yamira.poldosilva/Documents/UDEM/A25/IFT3395/kaggle_1_25-
10/.venv/lib/python3.13/site-packages/sklearn/linear_model/_logistic.py:1221:
UserWarning: l1_ratio parameter is only used when penalty is 'elasticnet'. Got
(penalty=l2)
  warnings.warn(
/Users/yamira.poldosilva/Documents/UDEM/A25/IFT3395/kaggle_1_25-
10/.venv/lib/python3.13/site-packages/sklearn/linear_model/_logistic.py:1221:
UserWarning: l1_ratio parameter is only used when penalty is 'elasticnet'. Got
(penalty=l2)
  warnings.warn(
/Users/yamira.poldosilva/Documents/UDEM/A25/IFT3395/kaggle_1_25-
10/.venv/lib/python3.13/site-packages/sklearn/linear_model/_logistic.py:1221:
UserWarning: l1_ratio parameter is only used when penalty is 'elasticnet'. Got
(penalty=l2)
  warnings.warn(
/Users/yamira.poldosilva/Documents/UDEM/A25/IFT3395/kaggle_1_25-
10/.venv/lib/python3.13/site-packages/sklearn/linear_model/_logistic.py:1221:
UserWarning: l1_ratio parameter is only used when penalty is 'elasticnet'. Got
(penalty=l2)
  warnings.warn(
/Users/yamira.poldosilva/Documents/UDEM/A25/IFT3395/kaggle_1_25-
10/.venv/lib/python3.13/site-packages/sklearn/linear_model/_logistic.py:1221:
UserWarning: l1_ratio parameter is only used when penalty is 'elasticnet'. Got
(penalty=l2)
  warnings.warn(
/Users/yamira.poldosilva/Documents/UDEM/A25/IFT3395/kaggle_1_25-
10/.venv/lib/python3.13/site-packages/sklearn/linear_model/_logistic.py:1221:
UserWarning: l1_ratio parameter is only used when penalty is 'elasticnet'. Got
(penalty=l2)
  warnings.warn(
/Users/yamira.poldosilva/Documents/UDEM/A25/IFT3395/kaggle_1_25-
10/.venv/lib/python3.13/site-packages/sklearn/linear_model/_logistic.py:1221:
UserWarning: l1_ratio parameter is only used when penalty is 'elasticnet'. Got
(penalty=l2)
  warnings.warn(
/Users/yamira.poldosilva/Documents/UDEM/A25/IFT3395/kaggle_1_25-
10/.venv/lib/python3.13/site-packages/sklearn/linear_model/_logistic.py:1221:
UserWarning: l1_ratio parameter is only used when penalty is 'elasticnet'. Got
(penalty=l1)
  warnings.warn(
/Users/yamira.poldosilva/Documents/UDEM/A25/IFT3395/kaggle_1_25-
10/.venv/lib/python3.13/site-packages/sklearn/linear_model/_logistic.py:1221:
UserWarning: l1_ratio parameter is only used when penalty is 'elasticnet'. Got
(penalty=l1)
  warnings.warn(
/Users/yamira.poldosilva/Documents/UDEM/A25/IFT3395/kaggle_1_25-
10/.venv/lib/python3.13/site-packages/sklearn/linear_model/_logistic.py:1221:
UserWarning: l1_ratio parameter is only used when penalty is 'elasticnet'. Got
(penalty=l1)
  warnings.warn(
```

```
/Users/yamira.poldosilva/Documents/UDEM/A25/IFT3395/kaggle_1_25-
10/.venv/lib/python3.13/site-packages/sklearn/linear_model/_logistic.py:1221:
UserWarning: l1_ratio parameter is only used when penalty is 'elasticnet'. Got
(penalty=l1)
  warnings.warn(
/Users/yamira.poldosilva/Documents/UDEM/A25/IFT3395/kaggle_1_25-
10/.venv/lib/python3.13/site-packages/sklearn/linear_model/_logistic.py:1221:
UserWarning: l1_ratio parameter is only used when penalty is 'elasticnet'. Got
(penalty=l2)
  warnings.warn(
/Users/yamira.poldosilva/Documents/UDEM/A25/IFT3395/kaggle_1_25-
10/.venv/lib/python3.13/site-packages/sklearn/linear_model/_logistic.py:1221:
UserWarning: l1_ratio parameter is only used when penalty is 'elasticnet'. Got
(penalty=l2)
  warnings.warn(
/Users/yamira.poldosilva/Documents/UDEM/A25/IFT3395/kaggle_1_25-
10/.venv/lib/python3.13/site-packages/sklearn/linear_model/_logistic.py:1221:
UserWarning: l1_ratio parameter is only used when penalty is 'elasticnet'. Got
(penalty=l1)
  warnings.warn(
/Users/yamira.poldosilva/Documents/UDEM/A25/IFT3395/kaggle_1_25-
10/.venv/lib/python3.13/site-packages/sklearn/linear_model/_logistic.py:1221:
UserWarning: l1_ratio parameter is only used when penalty is 'elasticnet'. Got
(penalty=l2)
  warnings.warn(
/Users/yamira.poldosilva/Documents/UDEM/A25/IFT3395/kaggle_1_25-
10/.venv/lib/python3.13/site-packages/sklearn/linear_model/_logistic.py:1221:
UserWarning: l1_ratio parameter is only used when penalty is 'elasticnet'. Got
(penalty=l2)
  warnings.warn(
/Users/yamira.poldosilva/Documents/UDEM/A25/IFT3395/kaggle_1_25-
10/.venv/lib/python3.13/site-packages/sklearn/linear_model/_logistic.py:1221:
UserWarning: l1_ratio parameter is only used when penalty is 'elasticnet'. Got
(penalty=l2)
  warnings.warn(
/Users/yamira.poldosilva/Documents/UDEM/A25/IFT3395/kaggle_1_25-
10/.venv/lib/python3.13/site-packages/sklearn/linear_model/_logistic.py:1221:
UserWarning: l1_ratio parameter is only used when penalty is 'elasticnet'. Got
(penalty=l2)
  warnings.warn(
/Users/yamira.poldosilva/Documents/UDEM/A25/IFT3395/kaggle_1_25-
10/.venv/lib/python3.13/site-packages/sklearn/linear_model/_logistic.py:1221:
UserWarning: l1_ratio parameter is only used when penalty is 'elasticnet'. Got
(penalty=l2)
  warnings.warn(

Accuracy: 0.3645
Recall Macro: 0.4127
```

F1 Macro: 0.2943

	precision	recall	f1-score	support
4	0.09	0.57	0.15	23
5	0.58	0.51	0.54	299
6	0.61	0.23	0.34	397
7	0.33	0.38	0.36	151
8	0.05	0.37	0.09	19
accuracy			0.36	889
macro avg	0.33	0.41	0.29	889
weighted avg	0.52	0.36	0.40	889



Meilleurs hyperparam RandomizedSearchCV :{'classifier__penalty': 'l2',
'classifier__l1_ratio': 0, 'classifier__C': 5}
Meilleur score en CV: 0.4400178972462666
Accuracy sur validation: 0.3644544431946007
Recall macro sur validation: 0.41274140335433945

[471]:
```
print("Optimisation k-NN...")
from sklearn.neighbors import KNeighborsClassifier
knn_pipeline = Pipeline([
    ('preprocessor', preprocessor),
    ('classifier', KNeighborsClassifier())
])

knn_hyperparam = {
    'classifier__n_neighbors': [3, 5, 7, 9, 11, 15, 21],
    'classifier__weights': ['uniform', 'distance'],
    'classifier__p': [1, 2],
    'classifier__leaf_size': [15, 30, 50],
    'classifier__metric': ['minkowski']
}

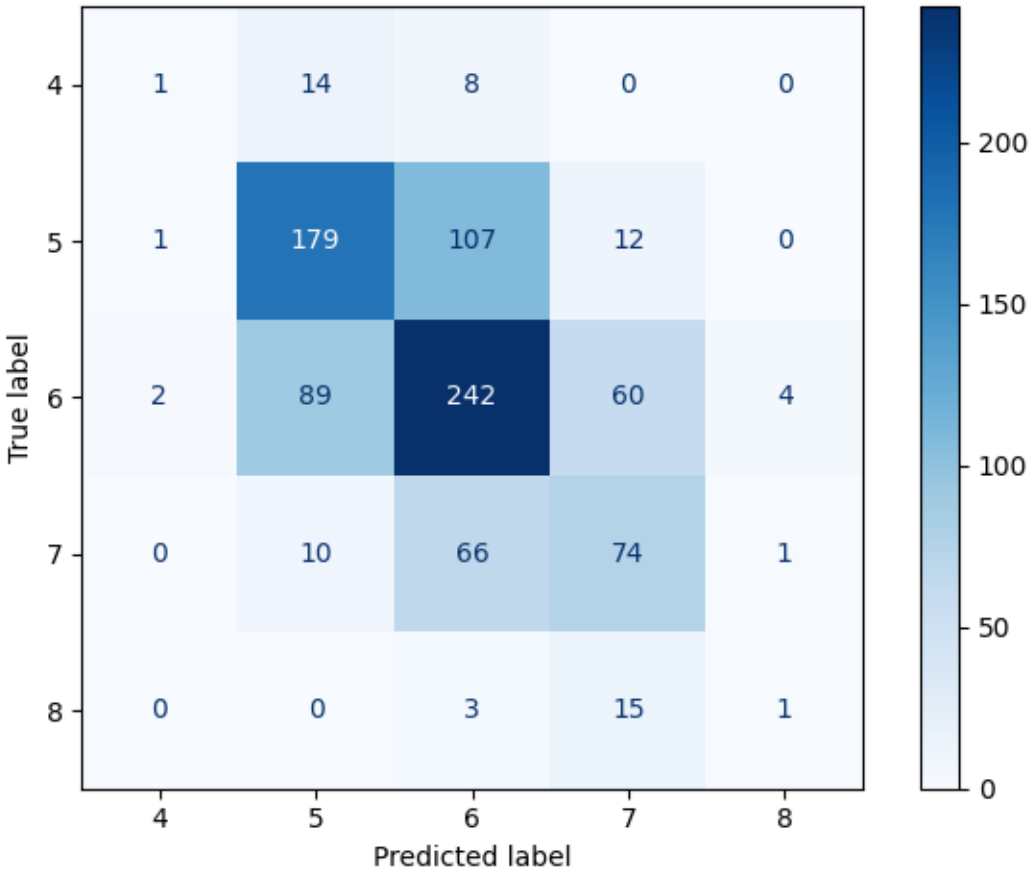
knn = RandomizedSearchCV(
    knn_pipeline,
    knn_hyperparam,
    n_iter=20,
    cv=5,
    scoring='recall_macro',
    n_jobs=-1,
    random_state=42
)

knn.fit(X_train, y_train)
y_pred_knn = knn.predict(X_val)

gscore(y_val, y_pred_knn, knn)
training_score(knn, y_val, y_pred_knn)
```

```
Optimisation k-NN…
Accuracy: 0.5591
Recall Macro: 0.3589
F1 Macro: 0.3645
```

	precision	recall	f1-score	support
4	0.25	0.04	0.07	23
5	0.61	0.60	0.61	299
6	0.57	0.61	0.59	397
7	0.46	0.49	0.47	151
8	0.17	0.05	0.08	19
accuracy			0.56	889
macro avg	0.41	0.36	0.36	889
weighted avg	0.55	0.56	0.55	889




```
Meilleurs hyperparam RandomizedSearchCV :{'classifier__weights': 'distance',
'classifier__p': 1, 'classifier__n_neighbors': 5, 'classifier__metric':
'minkowski', 'classifier__leaf_size': 30}
Meilleur score en CV: 0.3764419188303907
Accuracy sur validation: 0.5590551181102362
Recall macro sur validation: 0.3588820121506909
```

[472]:
```python
gb_proba = gb.best_estimator_.predict_proba(X_val)
lr_proba = lr.best_estimator_.predict_proba(X_val)
classes = lr.best_estimator_.classes_

def weighted_voting(w):
    if sum(w) == 0:
        return classes[gb_proba.argmax(1)]
    weights= (w[0]*gb_proba + w[1]*lr_proba) / sum(w)
    return classes[weights.argmax(1)]

ws = np.linspace(0, 20, 21)
best_score, best_weights = -1, None
```

```
for w in product(ws, repeat=2):

    temp = weighted_voting(w)
    score = recall_score(y_val, temp, average='macro')

    if score > best_score:
        best_score, best_weights = score, w

print(f"Les poids optimales pour [gb, lr] sont: {best_weights}")
```

```
Les poids optimales pour [gb, lr] sont: (np.float64(1.0), np.float64(16.0))
```

[473]:
```
voting_clf_final = VotingClassifier(
    estimators=[
        #('rf', rf.best_estimator_),
        ('gb', gb.best_estimator_),
        ('lr', lr.best_estimator_),
    ],
    voting='soft',
    weights=best_weights,
    n_jobs=-1
)

voting_clf_final.fit(X_train, y_train)

y_pred_final = voting_clf_final.predict(X_val)

gscore(y_val, y_pred_final, voting_clf_final)
```

```
/Users/yamira.poldosilva/Documents/UDEM/A25/IFT3395/kaggle_1_25-
10/.venv/lib/python3.13/site-packages/sklearn/linear_model/_logistic.py:1221:
UserWarning: l1_ratio parameter is only used when penalty is 'elasticnet'. Got
(penalty=l2)
  warnings.warn(

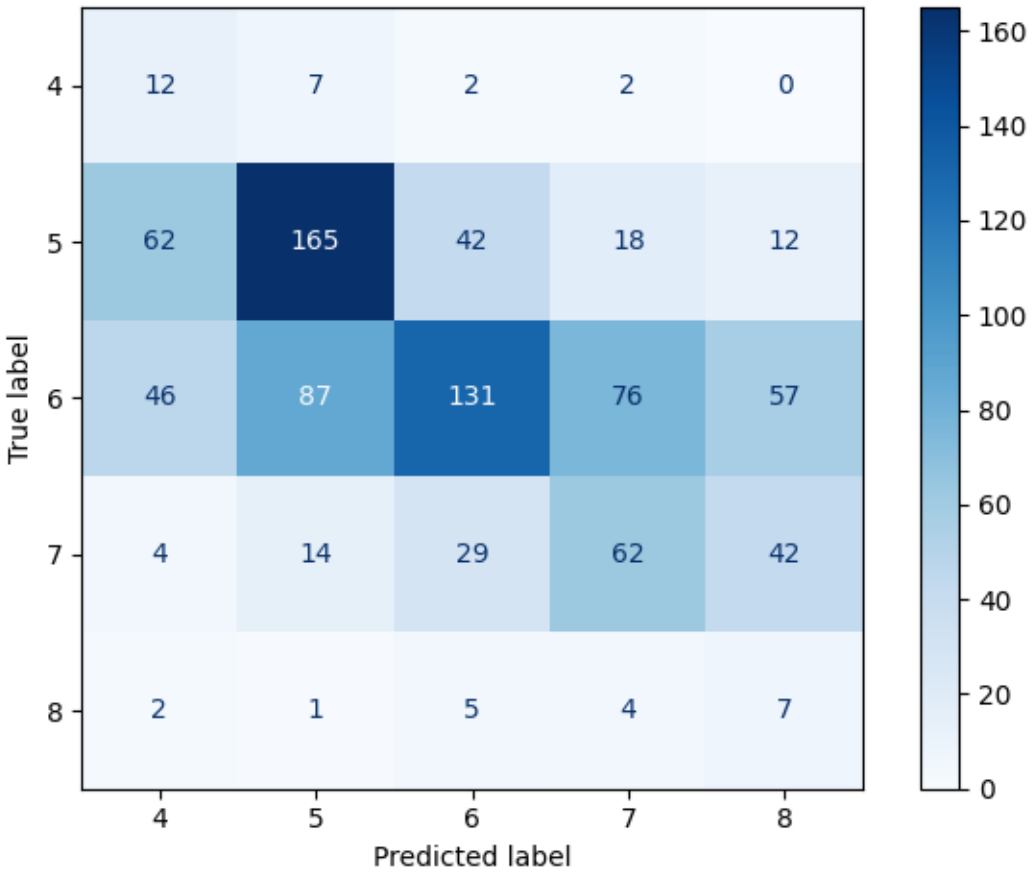
Accuracy: 0.4241
Recall Macro: 0.4365
F1 Macro: 0.3335

              precision    recall  f1-score   support

           4       0.10      0.52      0.16        23
           5       0.60      0.55      0.58       299
           6       0.63      0.33      0.43       397
           7       0.38      0.41      0.40       151
           8       0.06      0.37      0.10        19

    accuracy                           0.42       889
```

```
   macro avg       0.35      0.44      0.33       889
weighted avg       0.55      0.42      0.46       889
```



[474]:
```python
def get_best_weights(pp, classes):

    ws = np.linspace(0.0, 1.5, 31)
    best_score, best_weights = -1, (1.0, 1.0)

    for w4, w8 in product(ws, ws):

        adjusted = pp.copy()
        adjusted[:, classes == 4] *= w4
        adjusted[:, classes == 8] *= w8

        y_pred = classes[adjusted.argmax(1)]

        macro_recall = recall_score(y_val, y_pred, average="macro")
```

```
        if macro_recall > best_score:
            best_score, best_weights = macro_recall, (w4, w8)


    w4, w8 = best_weights
    print(f"Les poids optimales pour [w4, w8] sont {best_weights}")

    return w4,w8
```

[475]:
```
"""
Ajuste les poids des classes minoritaires
"""

pp = voting_clf_final.predict_proba(X_val).copy()
classes = voting_clf_final.classes_
w4,w8= get_best_weights(pp, classes)

pp[:, classes == 4] *= w4
pp[:, classes == 8] *= w8

y_pred_final = classes[pp.argmax(1)]

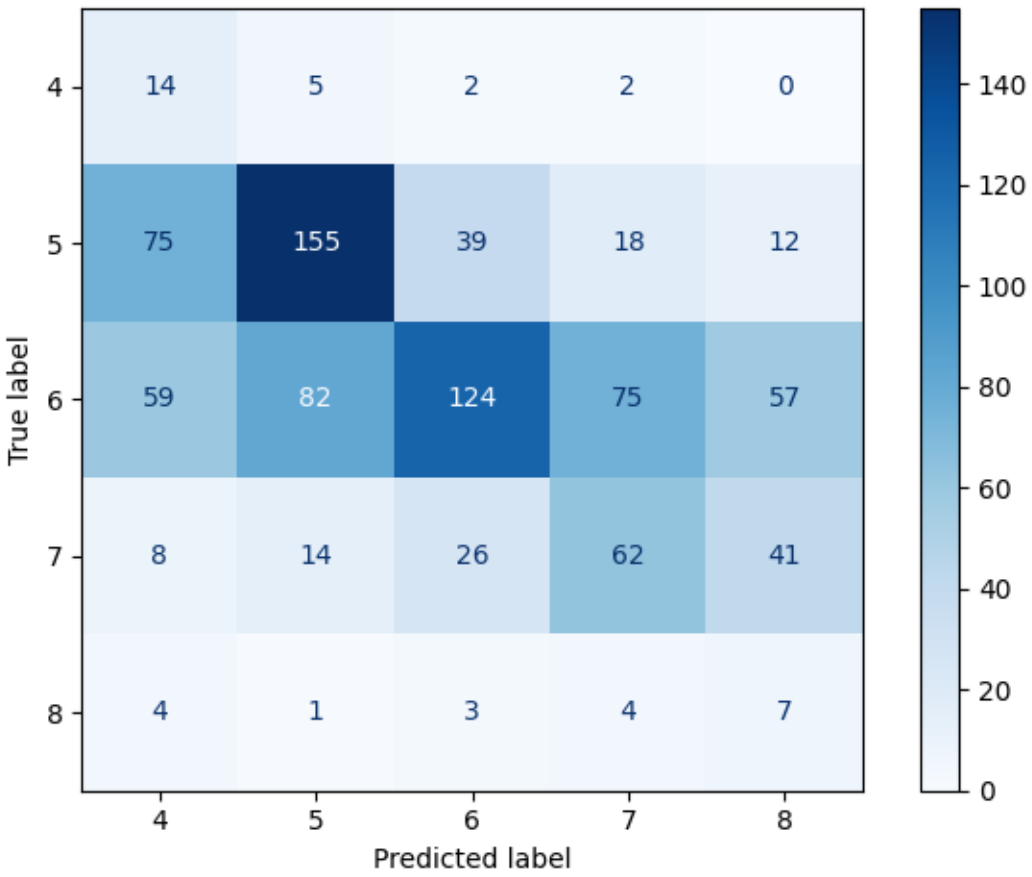
gscore(y_val, y_pred_final, voting_clf_final)
```

```
Les poids optimales pour [w4, w8] sont (np.float64(1.2000000000000002),
np.float64(1.0))
Accuracy: 0.4072
Recall Macro: 0.4437
F1 Macro: 0.3261

              precision    recall  f1-score   support

           4       0.09      0.61      0.15        23
           5       0.60      0.52      0.56       299
           6       0.64      0.31      0.42       397
           7       0.39      0.41      0.40       151
           8       0.06      0.37      0.10        19

    accuracy                           0.41       889
   macro avg       0.35      0.44      0.33       889
weighted avg       0.56      0.41      0.45       889
```



[476]:
```python
# Charger test data
test_df = pd.read_csv("data/test.csv", sep=";")

test_ids = test_df['id'].copy()

test_df = preprocessing_caracterisques(test_df, is_train=False)
test_df, _ = impute_values(test_df, is_train=False, stats=stats)
test_df = test_df.drop(columns=dropped_features, errors='ignore')

train_features = X.columns.tolist()
missing_cols = set(train_features) - set(test_df.columns)
extra_cols = set(test_df.columns) - set(train_features)

for col in missing_cols:
    test_df[col] = 0

test_df = test_df.drop(columns=list(extra_cols), errors='ignore')

X_test = test_df[train_features].copy()
```

```
assert X_test.shape[1] == X.shape[1], "Mismatch colonnes train/test"

proba_test = voting_clf_final.predict_proba(X_test)
classes = voting_clf_final.classes_

proba_test[:, classes == 4] *= w4
proba_test[:, classes == 8] *= w8

proba_test /= proba_test.sum(1, keepdims=True)

y_pred = classes[proba_test.argmax(axis=1)]

print("Distribution finale :")
print(pd.Series(y_pred).value_counts().sort_index())
```

```
Distribution finale :
4    110
5    120
6     87
7    113
8     81
Name: count, dtype: int64
```

[477]:
```
submission = pd.DataFrame({
    'id': test_ids,
    'quality': y_pred.astype(int)
})

assert len(submission) == len(test_ids),  "Nombre de prédictions incorrect!"
assert submission['id'].nunique() == len(submission), "IDs dupliqués!"
assert submission['quality'].isnull().sum() == 0, "Valeurs manquantes dans les␣
 ↪prédictions!"

filename = "ift3395_YAPS_MCS_V46.csv"
submission.to_csv(filename, index=False)

df1 = pd.read_csv("ift3395_YAPS_MCS_V42.csv")
df2 = pd.read_csv(filename)

comparison = df1.compare(df2)
print(comparison)
print("Nombre de différences :", len(comparison))
```

```
Empty DataFrame
Columns: []
Index: []
Nombre de différences : 0
```